

**UNIVERSIDADE DO VALE DO ITAJAÍ
CAMPUS ITAJAÍ**

PROJETO DE SISTEMAS DISTRIBUÍDOS: CHAT DISTRIBUÍDO

IAN CALLEGARI ARAGÃO, LUCAS LOSEKANN ROSA E LUCAS CARVALHO DE BORBA

Itajaí, 09 de Setembro de 2026

IAN CALLEGARI ARAGÃO, LUCAS LOSEKANN ROSA E LUCAS CARVALHO DE BORBA

PROJETO DE SISTEMAS DISTRIBUÍDOS: CHAT DISTRIBUÍDO

Trabalho da M1 do Curso de Ciência da Computação, da matéria de Sistemas Distribuídos da Universidade do Vale do Itajaí, ministrada pelo Professor Ramicés dos Santos Silva.

Itajaí, 09 de Setembro de 2026

RESUMO

Este relatório apresenta o desenvolvimento de um chat distribuído executado por processos independentes e comunicado exclusivamente por sockets TCP. A solução permite configurar até 15 nós a partir de um catálogo estático de endereços, mantém relógios vetoriais e utiliza um nó coordenador como sequenciador das mensagens de grupo. Cada mensagem de grupo recebe um número de sequência global e é difundida por unicast a todos os participantes; em cada receptor, um buffer somente libera sequências contíguas. Também foram implementados detecção de falhas por batimentos periódicos, reeleição do coordenador pelo algoritmo do Valentão e captura de estado global pelo algoritmo de Chandy–Lamport. A captura registra estados locais e mensagens em trânsito sem interromper o chat. Ensaios automatizados com sockets reais em uma única máquina, envolvendo 3, 8 e 15 processos independentes, confirmaram que todos os nós ativos entregaram as mensagens de grupo na mesma ordem e sem itens remanescentes no buffer. O snapshot foi concluído com 3 e 15 processos, e um teste controlado confirmou o registro de uma mensagem em trânsito. Em um ensaio de falha, o nó 2 foi eleito após a queda do coordenador de maior ID, recuperou as sequências 1 e 2 a partir dos sobreviventes e entregou a mensagem seguinte como sequência 3. Dessa forma, a ordem total permaneceu idêntica após a reeleição.

Palavras-chave: sistemas distribuídos; comunicação de grupo; ordem total; relógio vetorial; snapshot distribuído.

SUMÁRIO

1	INTRODUÇÃO	3
2	PROJETO E ARQUITETURA	4
2.1	Tema e escopo	4
2.2	Participação da equipe	4
2.3	Tecnologias utilizadas	5
2.4	Componentes da solução	5
2.5	Formato e fluxo das mensagens	6
2.6	Endereçamento	6
2.7	Interface e execução	7
3	ALGORITMOS DISTRIBUÍDOS	9
3.1	Relógio vetorial	9
3.2	Ordem total por sequenciador	9
3.2.1	Exemplo numérico com mensagens concorrentes	10
3.3	Monitoramento e eleição de líder	11
3.3.1	Continuidade da ordem após falha	12
3.4	Estado global por Chandy–Lamport	12
4	AVALIAÇÃO EXPERIMENTAL	14
4.1	Metodologia	14
4.2	Ordem global com 3, 8 e 15 nós	14
4.3	Captura de estado global	15
4.4	Inicialização, falha e reeleição	16
4.5	Mensagem privada	16
4.6	Limitações	17
4.7	Síntese dos requisitos	18
5	CONCLUSÃO	19

REFERÊNCIAS 20

1 INTRODUÇÃO

Sistemas distribuídos são formados por processos independentes que cooperam por troca de mensagens. Como não há memória nem relógio físico compartilhados, a observação de eventos e a manutenção de uma ordem comum exigem protocolos explícitos. Relógios lógicos preservam relações de precedência, enquanto relógios vetoriais permitem representar relações causais e identificar eventos concorrentes (LAMPORT, 1978; TANENBAUM; STEEN, 2007). Ainda assim, causalidade define apenas uma ordem parcial: mensagens concorrentes podem ser observadas em ordens diferentes por participantes distintos.

O trabalho propõe a implementação de uma aplicação distribuída com comunicação de grupo, ordem total, número configurável de nós, interface por nó e *snapshot* de um estado global consistente. Entre os temas oferecidos, escolheu-se o **chat distribuído**. Cada participante é um processo do sistema operacional, com estado privado e endereço próprio. A única informação compartilhada antes da execução é o catálogo estático de endereços; durante a execução, toda coordenação ocorre por mensagens de rede.

Para estabelecer uma sequência única das mensagens de grupo, foi escolhida a abordagem de **sequenciador**. Um coordenador atribui números globais monotônicos e redifunde as mensagens; relógios vetoriais permanecem anexados aos eventos para registrar a evolução causal. Como o coordenador é um ponto crítico, o protótipo emprega *health-checks* e o algoritmo do Valentão para eleição de um substituto. Para o estado global, emprega-se *Chandy-Lamport* sobre canais lógicos FIFO identificados e numerados na camada de aplicação. Essa opção permite registrar estados locais e mensagens em trânsito sem interromper os participantes.

O objetivo deste relatório é descrever a arquitetura, explicar seus algoritmos, decisões e registrar os resultados verificáveis. Por isso, além das funcionalidades demonstradas, são apresentadas e discutidas as limitações ainda existentes no projeto.

2 PROJETO E ARQUITETURA

2.1 TEMA E ESCOPO

A aplicação representa um bate-papo no qual cada nó corresponde a um participante. O escopo implementado prioriza o middleware: inicialização de múltiplos processos, serialização das mensagens, comunicação ponto a ponto, difusão ao grupo, registro das ordens local e global, relógio vetorial, monitoramento, eleição e *snapshot* distribuído. A interface é textual, suficiente para observar os estados internos relevantes ao experimento.

O número de nós é informado em tempo de execução. O arquivo `json/nodes.json` contém 15 entradas, e cada processo lê apenas as primeiras N . Portanto, a configuração entregue suporta de 1 a 15 nós sem alteração do código ou do arquivo de endereços, atendendo ao limite mínimo solicitado pelo roteiro.

2.2 PARTICIPAÇÃO DA EQUIPE

A divisão apresentada na Tabela 1 foi construída a partir do histórico de commits do repositório no *GitHub*, mas as atividades de integração, revisão, validação e criação do relatório foram conduzidas em conjunto. Ademais, a participação ativa do integrante, Lucas Carvalho de Borba, no desenvolvimento foi reduzida devido a sua presença no evento *LASC (Latin American Space Challenge)*.

Tabela 1 – Participação dos integrantes

Integrante			Principais contribuições
Ian Callegari Aragão			Configuração inicial; servidor e comunicação por sockets; tratamento de comandos; relógio vetorial; health-check, estados dos nós e ajustes na eleição.
Lucas Losekann Rosa			Interface de terminal e script de inicialização; mecanismo de ordenação e buffer de entrega; implementação e correções da eleição de líder; implementação do algoritmo de <i>Chandy-Lamport</i> .
Lucas	Carvalho	de Borba	Testes na aplicação, validação dos requisitos e suporte na construção do relatório.

2.3 TECNOLOGIAS UTILIZADAS

A implementação foi escrita em Python 3.10 ou superior. O projeto é gerenciado por `uv`, enquanto a comunicação, concorrência e serialização usam principalmente módulos da biblioteca padrão: `socket`, `threading`, `json`, `dataclasses`, `argparse` e `signal`. A rede é implementada diretamente sobre sockets TCP e a interface usa sequências ANSI no terminal.

2.4 COMPONENTES DA SOLUÇÃO

A Tabela 2 relaciona as principais unidades do projeto.

Tabela 2 – Componentes do protótipo

Componente	Responsabilidade
<code>lauch_node.py</code>	Lê os argumentos <code>-id</code> e <code>-count</code> , carrega o catálogo e instancia um nó.
<code>services/node.py</code>	Mantém relógios, históricos, buffer, sequências, estado dos pares, interface de comandos, heartbeat, eleição, recuperação da ordem e coordenação do snapshot.
<code>services/socket.py</code>	Abre o servidor TCP, aceita conexões e envia documentos JSON.
<code>services/terminal.py</code>	Mantém uma área de rolagem e uma linha fixa de entrada para a interface textual.
<code>models/node_dto.py</code>	Define o formato lógico das mensagens.
<code>models/snapshot_session.py</code>	Mantém os canais em gravação, estados de canal e relatórios de uma captura.
<code>json/nodes.json</code>	Catálogo estático com IDs, endereço IP e portas.
<code>start.sh</code>	Abre N processos em terminais Konsole separados.

Cada nó possui memória privada. Dentro do processo, três fluxos concorrentes são iniciados: escuta TCP, emissão periódica de batimentos (*heartbeat*) e verificação de saúde (*health-check*). A interface é executada no fluxo principal. Locks distintos protegem relógio vetorial, histórico local, contador do sequenciador, entrega, heartbeat e eleição.

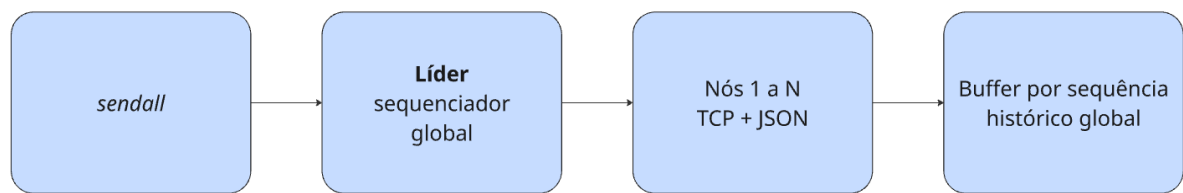


Figura 1 – Fluxo de uma mensagem de grupo

2.5 FORMATO E FLUXO DAS MENSAGENS

As mensagens são serializadas como objetos JSON derivados de `NodeDto`. Os campos comuns são: tipo, origem, horário, relógio vetorial e texto. Conforme a operação, também são informados destino, sequência local e sequência global. Os tipos declarados incluem mensagem privada, pedido de mensagem de grupo, mensagem de grupo ordenada, heartbeat, confirmação de heartbeat, eleição, coordenador, atualização da lista de nós, marcador e relatório de snapshot, pedido e resposta de estado da ordem global. Os campos `channel_origin` e `channel_sequence` identificam o emissor TCP e a posição da mensagem no canal lógico sem confundir-lo com o autor original de uma mensagem retransmitida pelo líder.

Em cada envio, `Socket.send` abre uma conexão TCP, transmite um documento JSON e fecha a conexão. O receptor aceita a conexão, lê blocos de até 65.536 bytes até o fechamento do emissor e então delega o documento completo ao nó (Python Software Foundation, 2026). A escolha por unicast TCP foi motivada pela entrega confiável e pela simplicidade de endereçamento em uma única máquina. Para difusão, o líder itera sobre o catálogo e envia uma cópia para cada participante. Em comparação ao multicast UDP, essa solução evita implementar confirmação e retransmissão de datagramas.

Além disso, o multicast UDP não oferece, por si só, um mecanismo específico para comunicação privada entre dois participantes, sendo necessária a implementação de uma lógica adicional para distinguir e encaminhar mensagens privadas. Com TCP unicast, essa distinção é obtida naturalmente pelo endereço do destinatário da conexão, não sendo necessário implementar um mecanismo adicional de comunicação privada. Como contrapartida, a difusão realiza N entregas por mensagem de grupo e concentra esse custo no coordenador.

2.6 ENDEREÇAMENTO

Todos os ensaios foram executados pela interface de loopback, de modo que a rede não sai da máquina local. Cada nó possui uma porta exclusiva, conforme a

Tabela 3. O servidor escuta em 0.0.0.0, enquanto o catálogo usa 127.0.0.1 como destino.

Tabela 3 – Catálogo de endereços da simulação

Nó	IP	Porta	Nó	IP	Porta	Nó	IP	Porta
1	127.0.0.1	5001	6	127.0.0.1	5006	11	127.0.0.1	5011
2	127.0.0.1	5002	7	127.0.0.1	5007	12	127.0.0.1	5012
3	127.0.0.1	5003	8	127.0.0.1	5008	13	127.0.0.1	5013
4	127.0.0.1	5004	9	127.0.0.1	5009	14	127.0.0.1	5014
5	127.0.0.1	5005	10	127.0.0.1	5010	15	127.0.0.1	5015

2.7 INTERFACE E EXECUÇÃO

O script automático depende de Linux, `bash`, `setsid`, `Konsole` e `uv`. Na raiz do repositório, a inicialização de três, oito ou quinze nós é feita por:

```
./start.sh --count <quantidade>
```

Também é possível abrir terminais manualmente e executar um processo em cada um:

```
uv run python lauch_node.py --id <identificador> --count <quantidade>
```

Os comandos da interface são descritos na Tabela 4.

Tabela 4 – Comandos disponíveis em cada nó

Comando	Função
<code>send ID mensagem</code>	Envia uma mensagem privada ao endereço do ID.
<code>sendall mensagem</code>	Solicita ao líder a ordenação e difusão ao grupo.
<code>local</code>	Exibe os envios produzidos localmente e sua sequência.
<code>global</code>	Exibe as mensagens de grupo já entregues em ordem total.
<code>status</code>	Exibe líder, vetor, próxima sequência e históricos.
<code>list</code>	Exibe endereço e estado ativo/inativo dos participantes.
<code>snapshot</code>	Inicia <i>Chandy-Lamport</i> e exibe o estado global após receber os fragmentos dos nós ativos.
<code>help</code>	Lista os comandos disponíveis.
<code>exit</code>	Encerra o processo.

No comando `send`, somente o emissor registra o evento em sua ordem local. O documento `PRIVATE_MESSAGE` é encaminhado ao endereço do destino, que combina

o vetor recebido com o seu relógio, incrementa a própria posição e exibe a mensagem com o rótulo `[PRIVADA]`. Como se trata de unicast, ela não recebe sequência global nem aparece no histórico dos demais participantes.

3 ALGORITMOS DISTRIBUÍDOS

3.1 RELÓGIO VETORIAL

Cada nó mantém um vetor V de tamanho N . A posição $V[i]$ representa o conhecimento local sobre os eventos do nó i . Ao criar uma mensagem local no nó i , a implementação executa

$$V[i] \leftarrow V[i] + 1 \quad (1)$$

e anexa uma cópia do vetor à mensagem. Ao processar um vetor recebido V_m , o nó calcula, componente a componente,

$$V[j] \leftarrow \max(V[j], V_m[j]), \quad 1 \leq j \leq N, \quad (2)$$

e depois incrementa sua própria posição. Essa política é compatível com a interpretação de recepção como evento local. O uso de vetores permite registrar mais informação causal que um relógio escalar (COULOURIS et al., 2011).

Na abordagem adotada, o vetor não decide a ordem total. Ele acompanha pedidos e mensagens ordenadas para permitir inspeção causal, enquanto o número fornecido pelo sequenciador é a autoridade para entrega. Portanto, a solução corresponde à abordagem B do roteiro.

3.2 ORDEM TOTAL POR SEQUENCIADOR

O líder inicial é determinado como o maior ID ativo do catálogo. Dessa forma, o critério do algoritmo do Valentão também é aplicado na inicialização, sem depender de uma primeira falha para selecionar o participante de maior prioridade. Uma mensagem de grupo percorre as etapas a seguir:

1. O emissor incrementa sua posição no relógio vetorial, incrementa o contador de ordem local e armazena o envio no histórico local.
2. Se o emissor não é o líder, abre uma conexão TCP com o coordenador e envia um `GROUP_MESSAGE_REQUEST`. Se é o líder, chama diretamente a rotina de sequenciamento.

3. O líder protege `next_global_sequence` com um lock, atribui seu valor à mensagem e incrementa o contador.
4. A mensagem passa a ter o tipo `ORDERED_GROUP_MESSAGE` e é enviada individualmente a todos os nós. A cópia do próprio líder é tratada localmente.
5. Cada receptor indexa a mensagem por sua sequência no buffer. Sequências menores que a próxima esperada são descartadas como duplicatas antigas.
6. Enquanto a próxima sequência esperada estiver presente, o receptor a remove do buffer, atualiza o relógio vetorial, acrescenta a mensagem ao histórico global e avança a sequência esperada.

O buffer é necessário porque conexões TCP diferentes não estabelecem ordem entre si. Assim, mesmo que a mensagem 4 chegue antes da 3, ela permanece retida até que a lacuna seja preenchida. Como todos recebem os mesmos números e aplicam a mesma regra de liberação, os históricos convergem para a mesma ordem.

3.2.1 Exemplo numérico com mensagens concorrentes

Considere três nós com vetores inicialmente iguais a $[0, 0, 0]$. O nó 2 cria a mensagem A e a carimba com $[0, 1, 0]$; concorrentemente, o nó 3 cria B com $[0, 0, 1]$. Nenhum vetor é componente a componente menor que o outro, portanto A e B são concorrentes. Suponha que o líder receba B antes de A. Ele atribui os números globais mostrados na Tabela 5.

Tabela 5 – Sequenciamento de duas mensagens concorrentes

Mensagem	Origem	Vetor	Ordem local	Ordem global
B	3	$[0, 0, 1]$	1	1
A	2	$[0, 1, 0]$	1	2

Se A, de sequência 2, chegar primeiro ao nó 1, ela aguarda no buffer. Quando B chega com sequência 1, o nó entrega B e, em seguida, A. O mesmo ocorre no nó 2:

$$H_1 = H_2 = \langle (1, B), (2, A) \rangle. \quad (3)$$

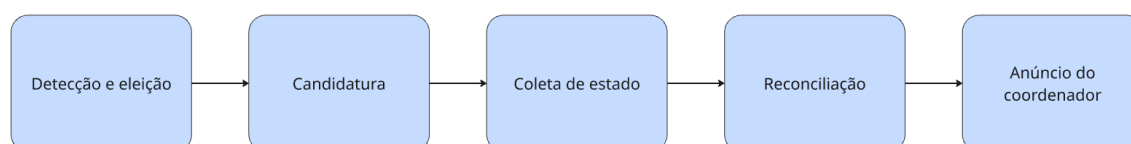
O exemplo mostra que o vetor identifica a concorrência, mas é a decisão do sequenciador que transforma as mensagens em uma ordem total.

3.3 MONITORAMENTO E ELEIÇÃO DE LÍDER

O coordenador envia uma mensagem `HEARTBEAT` a cada nó ativo a cada cinco segundos. O receptor responde com `HEARTBEAT_ACK`. O líder registra o instante da última confirmação, enquanto os seguidores registram o último batimento do coordenador. O limiar base é de dez segundos e a inativação ou eleição ocorre após três verificações vencidas. Quando o líder considera um participante inativo, difunde uma atualização da lista.

A eleição segue o algoritmo do Valentão, descrito resumidamente na Figura 2. Seu passo a passo na implementação é:

Figura 2 – Fluxograma resumido do funcionamento da eleição



1. O nó que detecta a indisponibilidade marca o líder anterior como inativo e envia `ELECTION` a cada nó ativo de ID maior.
2. Uma conexão bem-sucedida é tratada como indicação de que o nó superior continuará a eleição. Ao receber a solicitação, todo nó de ID superior inicia sua própria rodada.
3. Um participante sem nó superior acessível torna-se candidato a coordenador, mas ainda não libera novas sequências.
4. O candidato envia `ORDER_STATE_REQUEST` aos sobreviventes. Cada resposta contém histórico global, buffer e próxima entrega esperada.
5. O candidato une as mensagens pelo número global, rejeita conflitos ou lacunas, retransmite o prefixo contíguo a todos e define a próxima sequência como o maior número recuperado mais um.
6. Somente após a recuperação, o vencedor difunde `COORDINATOR`; os receptores atualizam `leader_id`, encerram a eleição e reenviam pedidos que haviam falhado ao acessar o líder anterior.

Consequentemente, o maior ID ativo alcançado vence, conforme a propriedade do algoritmo do Valentão (TANENBAUM; STEEN, 2007). A implementação simplifica o protocolo: não há mensagem `OK` separada nem timeout específico para aguardar o anúncio; o sucesso da conexão de eleição exerce o papel da confirmação.

3.3.1 Continuidade da ordem após falha

Durante a recuperação, o novo coordenador permanece com `leader_ready = False`; solicitações locais que chegarem nesse intervalo ficam em uma fila. Para reconstruir a ordem, ele considera tanto mensagens já entregues quanto mensagens ainda presentes nos buffers dos nós ativos. Cópias com o mesmo número devem possuir origem, sequência local, horário, vetor e conteúdo idênticos. Qualquer divergência é tratada como conflito e impede a abertura do sequenciador.

Se a união contém o prefixo $1, 2, \dots, k$, o candidato retransmite essas mensagens em ordem. Receptores que já as entregaram descartam as duplicatas; receptores atrasados preenchem suas lacunas e avançam. Em seguida, o contador do líder é ajustado para $k + 1$. Um pedido cujo envio ao líder antigo falhou é mantido no nó de origem e reenviado automaticamente após `COORDINATOR`. Assim, a primeira mensagem nova recebe $k + 1$, sem reiniciar a numeração nem divergir dos históricos existentes.

3.4 ESTADO GLOBAL POR CHANDY–LAMPORT

Um estado global reúne os estados locais e as mensagens em trânsito de modo compatível com a causalidade. O algoritmo de Chandy–Lamport é uma escolha adequada porque não exige relógio físico sincronizado nem a interrupção da aplicação; por meio de marcadores, ele registra um corte consistente da execução (CHANDY; LAMPORT, 1985).

O comando `snapshot`, disponível em qualquer nó ativo, executa as seguintes etapas:

1. O iniciador cria um identificador formado por seu ID e pelo tempo local, fixa o conjunto de participantes ativos e registra seu estado. O registro inclui relógio vetorial, líder, históricos local e global, buffer de entrega, próximas sequências e lista de nós ativos. Em seguida, envia um marcador a todos os canais de saída.
2. Ao receber o primeiro marcador, cada nó registra o próprio estado, marca o canal de chegada como vazio e encaminha marcadores aos demais nós.
3. Até receber o marcador de cada origem, o nó registra como estado daquele canal as mensagens de aplicação recebidas após seu estado local.
4. Quando todos os canais de entrada são fechados por marcadores, o nó envia um `SNAPSHOT_REPORT` ao iniciador. Este agrega um relatório de cada participante e exibe o estado global no terminal.

O transporte abre uma conexão TCP nova para cada documento. Como TCP só garante FIFO dentro de uma mesma conexão, a implementação acrescenta uma camada de canal lógico. Cada emissor TCP mantém, para cada destino, um contador protegido por lock e anexa `channel_origin` e `channel_sequence`. O receptor mantém a próxima posição esperada por emissor e retém conexões adiantadas até preencher as lacunas. Assim, mensagens de aplicação e marcadores são entregues em FIFO no canal lógico, que é a premissa necessária ao algoritmo.

É importante distinguir a origem de aplicação da origem de transporte. Quando o líder retransmite uma mensagem escrita pelo nó 3, o campo `origin` continua igual a 3, mas `channel_origin` identifica o líder que realizou a conexão. O estado de canal do snapshot usa o segundo campo. Sem essa separação, sequências do líder seriam atribuídas incorretamente aos autores originais e criariam lacunas fictícias.

Ao concluir, o iniciador imprime uma linha por nó com líder observado, relógio vetorial, quantidade de eventos locais, sequências globais entregues e próxima entrega esperada. Canais não vazios são apresentados com origem, destino, quantidade e tipos das mensagens em trânsito. O algoritmo não bloqueia o envio de mensagens da aplicação durante a captura.

4 AVALIAÇÃO EXPERIMENTAL

4.1 METODOLOGIA

Foram executados ensaios temporários sobre o código do repositório, sem modificar a implementação. Cada nó foi iniciado pelo sistema operacional como um processo separado, usando o ponto de entrada `lauch_node.py`, o catálogo original e as portas 5001 a 5015 em `127.0.0.1`. Os comandos foram escritos na entrada padrão dos processos em rápida sucessão e percorreram sockets TCP reais. Depois das entregas, o comando `global` foi solicitado em todos os terminais; as tuplas (sequência global, origem, texto) foram extraídas das saídas e comparadas. Todos os processos encerraram normalmente.

Foi adicionada uma suíte reproduzível baseada no módulo `unittest` da biblioteca padrão. Os arquivos `tests/test_snapshot.py`, `tests/test_private_message.py`, `tests/test_order_recovery.py` e `tests/test_initial_leader.py` validam a reordenação FIFO do canal lógico, forçam uma mensagem a ficar em trânsito durante uma captura, verificam a separação entre mensagens privadas e de grupo, simulam recuperação após difusão parcial e confirmam a seleção inicial do maior ID ativo. A execução é feita por:

```
uv run python -m unittest discover -s tests -v
```

4.2 ORDEM GLOBAL COM 3, 8 E 15 NÓS

Os três cenários foram concluídos com todos os históricos idênticos e todos os buffers vazios, conforme a Tabela 6. No ensaio com três nós, cada participante emitiu uma mensagem. Nos ensaios com oito e quinze, cinco nós emitiram simultaneamente.

Tabela 6 – Resultados da difusão e entrega ordenada

Nós	Envios	Todos receberam	Históricos idênticos	Buffers vazios
3	3	Sim	Sim	Sim
8	5	Sim	Sim	Sim
15	5	Sim	Sim	Sim

A saída resumida do cenário de 15 nós foi:

```
GLOBAL 1 | Node 1 | processo-1
GLOBAL 2 | Node 3 | processo-3
GLOBAL 3 | Node 2 | processo-2
```

```
GLOBAL 4 | Node 4 | processo-4
GLOBAL 5 | Node 5 | processo-5
```

```
comparacao dos 15 historicos: IDENTICOS
```

```
buffers restantes: [[], [], [], [], [], [], [], [], [], [], [], [], [], [], [],
[]]
```

Os pedidos dos nós 2 e 3 foram emitidos em rápida sucessão por processos independentes, mas o líder aceitou o pedido do nó 3 primeiro. Todos os participantes observaram a mesma inversão. Isso é uma evidência direta de que a ordem exibida foi determinada pela sequência global e não por uma suposição local sobre a ordem de emissão.

4.3 CAPTURA DE ESTADO GLOBAL

O snapshot foi validado em três níveis. No teste unitário controlado, o nó 1 registrou seu estado e enviou o primeiro marcador, que foi deliberadamente retido. Antes de o nó 2 receber esse marcador, ele enviou uma mensagem ao nó 1. A mensagem foi entregue ao iniciador antes do marcador de retorno. O resultado foi um corte consistente: o estado local do nó 2 continha o evento de envio, o estado local do nó 1 ainda não continha a recepção e o canal $2 \rightarrow 1$ continha exatamente essa mensagem em trânsito.

Depois, o comando `snapshot` foi executado com sockets reais e processos independentes. A Tabela 7 apresenta os resultados. No cenário com três nós, o nó 2 iniciou a captura depois de duas mensagens de grupo. No cenário com quinze nós, o nó 8 iniciou a captura depois de cinco mensagens.

Tabela 7 – Resultados da captura de estado global

Nós	Iniciador	Relatórios	Snapshot concluído	Processos encerrados
3	2	3	Sim	Normalmente
15	8	15	Sim	Normalmente

No ensaio com quinze nós, todos os relatórios exibiram o histórico global [1, 2, 3, 4, 5]. A verificação resumida produziu:

```
snapshot concluído: SIM
```

```
iniciador: Node 8
```

```
relatorios recebidos: 15
```

```
nodes: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
```

```
historicos globais identicos: SIM
```

```
historico de referencia: [1, 2, 3, 4, 5]
```

Esses ensaios demonstram tanto a conclusão do protocolo na escala exigida quanto o aspecto essencial de consistência: uma mensagem que cruza o corte não desaparece nem é contada como recebida antes de seu envio, pois permanece registrada no estado do canal.

4.4 INICIALIZAÇÃO, FALHA E REELEIÇÃO

A seleção inicial também foi verificada diretamente: com três processos, todos identificaram o nó 3 como coordenador; com quinze processos, todos identificaram o nó 15. Portanto, a inicialização aplica o mesmo critério de prioridade da eleição do Valentão, em vez de fixar o nó 1.

No cenário de falha com três processos, duas mensagens foram entregues nas sequências 1 e 2. Em seguida, o coordenador de maior ID, nó 3, foi encerrado abruptamente por `SIGKILL`. O nó 1 tentou enviar uma terceira mensagem; a falha de conexão fez com que ele preservasse o pedido e iniciasse a eleição. Os nós 1 e 2 elegeram o nó 2, que consultou os estados disponíveis, restaurou o prefixo $[1, 2]$ e definiu 3 como próxima sequência. Depois do anúncio, o nó 1 reenviou automaticamente o pedido pendente, entregue como sequência 3 nos dois sobreviventes.

Tabela 8 – Ensaio de troca do sequenciador

Observação	Resultado
Histórico antes da falha	$[1, 2]$ em todos os nós
Líder inicial entre os nós 1, 2 e 3	Nó 3
Líder eleito entre os nós 1 e 2	Nó 2
Prefixo recuperado pelo novo líder	$[1, 2]$
Próxima sequência restaurada	3
Pedido que detectou a falha	Reenviado automaticamente
Históricos finais dos sobreviventes	$[1, 2, 3]$ e $[1, 2, 3]$

Também foi executado um teste controlado de difusão parcial. Um sobrevivente possuía $[1, 2]$, enquanto o futuro líder possuía apenas $[1]$. A resposta de estado permitiu que o novo coordenador recuperasse a mensagem 2 e a retransmitisse antes de criar a sequência 3. Ambos terminaram com $[1, 2, 3]$. Os dois cenários demonstram continuidade e convergência da ordem total após falha do sequenciador.

4.5 MENSAGEM PRIVADA

O unicast privado foi validado inicialmente com um teste unitário de dois nós. O nó 1 enviou a mensagem ao nó 2, que atualizou seu relógio de $[0, 0]$ para $[1, 1]$ e

produziu exatamente uma saída [PRIVADA]. O evento permaneceu no histórico local do emissor, enquanto os históricos local e global do receptor não foram indevidamente alterados.

Em seguida, dois processos independentes trocaram a mensagem `mensagem-reservada` pelas portas 5001 e 5002. A mensagem apareceu uma vez no destino e nenhuma vez no emissor. Uma mensagem de grupo enviada como controle foi entregue globalmente nos dois processos e não recebeu o rótulo privado. O mesmo teste confirmou que mensagens de grupo remotas atualizam o vetor apenas uma vez.

4.6 LIMITAÇÕES

Além dos resultados anteriores, a versão atual possui as seguintes limitações:

- **Falha durante snapshot:** o conjunto de participantes é fixado no início e não há timeout ou repetição de marcador. Se um participante falhar durante a captura, o iniciador continuará aguardando seu relatório.
- **Custo do snapshot:** na topologia completamente conectada, cada nó envia um marcador aos demais, resultando em $N(N - 1)$ marcadores, além dos relatórios enviados ao iniciador.
- **Recuperação da ordem:** o candidato aguarda respostas por três segundos e prossegue apenas com os participantes responsivos. O protocolo pressupõe que ao menos um sobrevivente possua cada sequência que chegou a ser difundida; se detectar conflito ou lacuna na união, não abre o sequenciador. A retransmissão percorre todo o histórico conhecido, cujo custo cresce com o tempo de execução.
- **TCP:** o fechamento da conexão delimita cada documento e o receptor acumula blocos até o fim do fluxo, mas não há autenticação, criptografia ou repetição automática após falha de envio.
- **Escalabilidade:** uma difusão requer uma conexão por destinatário e é feita sequencialmente pelo líder. O catálogo entregue termina no nó 15, e `start.sh` não valida pedidos acima desse limite.
- **Portabilidade:** a inicialização automática depende do terminal Konsole e de utilitários típicos de Linux. A execução manual continua possível em outros ambientes compatíveis com Python.
- **Testes:** a suíte versionada cobre FIFO, snapshot, mensagem privada e recuperação da ordem, mas os ensaios em maior escala ainda são executados

externamente. Também não foram realizados ensaios entre máquinas físicas, sob perda de conexão intermitente ou com mensagens de grande volume.

4.7 SÍNTESE DOS REQUISITOS

A Tabela 9 resume a cobertura verificada por inspeção e pelos ensaios descritos neste capítulo.

Tabela 9 – Cobertura dos requisitos funcionais

Req.	Situação	Evidência
R1	Atendido	Difusão TCP entregue a todos em 3, 8 e 15 nós.
R2	Atendido	Mensagens privadas e de grupo foram entregues por TCP.
R3	Atendido	Inicialização e entrega verificadas com 3, 8 e 15 processos.
R4	Atendido	Históricos são idênticos no caminho normal e após queda, recuperação e substituição do sequenciador.
R5	Atendido	Terminal oferece unicast, grupo, ordens local e global, estado, lista de nós e snapshot.
R6	Atendido	Chandy–Lamport registra estados locais e canais; o comando foi concluído com 3 e 15 processos e registrou mensagem em trânsito.
R7	Atendido	Itens documentais do roteiro são cobertos neste relatório.
R8	Atendido	O maior ID ativo é eleito, recupera o prefixo global e anuncia sua coordenação antes de retomar os pedidos.

5 CONCLUSÃO

O protótipo demonstra os elementos centrais de comunicação de grupo e ordenação total em um chat distribuído. Cada participante possui processo, porta e estado próprios; mensagens de grupo atravessam sockets TCP e recebem uma sequência única do coordenador. O buffer por lacunas faz com que mensagens recebidas fora de ordem sejam entregues somente quando todas as posições anteriores estão disponíveis. Nos ensaios com 3, 8 e 15 nós, todos os participantes terminaram com históricos globais idênticos e buffers vazios.

O relógio vetorial acompanha as mensagens e torna visível a evolução causal, mas a ordem total é decidida pelo sequenciador, como previsto pela abordagem B. O monitoramento por heartbeat e a eleição do Valentão conseguiram selecionar o maior ID ativo após a queda do líder em um cenário com três nós. A arquitetura, portanto, oferece uma base concreta para estudar comunicação, relógios lógicos, difusão e coordenação. O comando `snapshot` acrescenta uma fotografia consistente dos estados locais e canais sem suspender a aplicação. Nos ensaios, a captura agregou 3 e 15 relatórios, e o cenário controlado preservou corretamente uma mensagem em trânsito.

O R6 é atendido pela implementação de Chandy–Lamport, apoiada pela numeração FIFO dos canais lógicos e pela agregação automática dos fragmentos no iniciador. A continuidade da ordem total também foi demonstrada: depois da queda do coordenador inicial, nó 3, o nó 2 recuperou o prefixo global, adotou a próxima sequência correta e entregou o pedido que havia detectado a falha. A comunicação privada foi validada separadamente e não interfere na ordem global das mensagens de grupo.

Os requisitos funcionais do roteiro foram demonstrados nos cenários executados. Como evolução, recomenda-se reduzir o custo de retransmitir todo o histórico na recuperação, adicionar persistência opcional e tratar a falha de um participante durante um snapshot com timeout e nova configuração de membros. Essas limitações não alteraram a convergência nos testes apresentados, mas delimitam o comportamento do protótipo sob falhas sucessivas ou execuções prolongadas.

REFERÊNCIAS

CHANDY, K. M.; LAMPORT, L. Distributed snapshots: Determining global states of distributed systems. **ACM Transactions on Computer Systems**, v. 3, n. 1, p. 63–75, 1985.

COULOURIS, G.; DOLLIMORE, J.; KINDBERG, T.; BLAIR, G. **Distributed Systems: Concepts and Design**. 5. ed. Boston: Addison-Wesley, 2011.

LAMPORT, L. Time, clocks, and the ordering of events in a distributed system. **Communications of the ACM**, v. 21, n. 7, p. 558–565, 1978.

Python Software Foundation. **socket — Low-level networking interface**. [S.l.], 2026. Disponível em: <<https://docs.python.org/3/library/socket.html>>. Acesso em: 8 set. 2026.

TANENBAUM, A. S.; STEEN, M. V. **Distributed Systems: Principles and Paradigms**. 2. ed. Upper Saddle River: Pearson Prentice Hall, 2007.